

Python Practice Workbook

Module 1: Anatomy & Smoothing

Trend, Seasonality, ACF, Portmanteau, Transformations

Time Series Analytics T6725 — M.Sc. Data Science Sem IV
Symbiosis International (Deemed) University

Standard Setup Block — Run this first in every Colab session

```
!pip install yfinance statsmodels arch -q
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

Problem Distribution

Type	Count	Difficulty Spread
Fill in the Blank	6	2E, 2M, 2H
Find the Error	6	2E, 2M, 2H
Get the Order Right	4	2E, 2M
Match the Following	4	2E, 2M
Multiple Choice	6	2E, 2M, 2H
Write from Scratch	5	1E, 2M, 2H
Predict the Output	5	2E, 2M, 1H
Short Answer	3	1E, 1M, 1H
Exam Style	1	Full exam
TOTAL	40	

Section A

Fill in the Blank

M1.001 Fill in the Blank **EASY**

Topic: Downloading data and extracting closing prices

Fill in the **three blanks** so the code downloads Bitcoin data from 2019 and plots its closing price.

```

1  # Download Bitcoin data
2  btc = yf._____("BTC-USD", start="_____",
3               end="2024-12-31")
4
5  # Extract closing price
6  close = btc["_____"]
7
8  # Plot
9  close.plot(title="Bitcoin Closing Price")
10 plt.xlabel("Date")
11 plt.ylabel("Price (USD)")
12 plt.show()

```

Solution

① Correct Code

```

1  !pip install yfinance statsmodels arch -q
2  import yfinance as yf
3  import pandas as pd
4  import numpy as np
5  import matplotlib.pyplot as plt
6  import warnings
7  warnings.filterwarnings('ignore')
8
9  # Download Bitcoin data
10 btc = yf.download("BTC-USD", start="2019-01-01",
11                  end="2024-12-31")
12
13 # download() fetches OHLCV data
14
15 # Extract closing price
16 close = btc["Close"] # "Close" column has the daily
17                        closing price
18
19 # Plot
20 close.plot(title="Bitcoin Closing Price")
21 plt.xlabel("Date")
22 plt.ylabel("Price (USD)")
23 plt.show()

```

② Plain English

`yf.download()` fetches historical OHLCV data from Yahoo Finance. The "Close" column gives you the price at which BTC traded at end of each day — this is the raw time series you will analyze throughout this course.

③ Common Mistake

Writing "close" (lowercase) instead of "Close" — Python is case-sensitive and pandas column names must match exactly. Also, many students forget the `start=` keyword argument and pass just the string positionally — always use keyword arguments with `yfinance`.

④ Real World Link

The Bitcoin closing price series is your playground for the entire course. Every spike you see — the 2021 bull run, the 2022 crash, the 2024 halving rally — is a signal buried in this single column waiting to be decomposed.

M1.002 Fill in the Blank **EASY**

Topic: Computing a Simple Moving Average

Fill in the blanks to compute and plot a **30-day moving average** of MRF share prices.

```
1 mrf = yf.download("_____", start="2019-01-01",
2   end="2024-12-31")
3
4 close = mrf["Close"]
5
6
7 # Compute 30-day moving average
8 ma30 = close.rolling(window=_____._____.())
9
10 # Plot both
11 close.plot(label="MRF Close", alpha=0.5)
12 ma30.plot(label="30-Day MA", color="red")
13 plt.legend()
14 plt.title("MRF -- Price vs 30-Day Moving Average")
15 plt.show()
```

Solution

① Correct Code

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import matplotlib.pyplot as plt
4 import warnings
5 warnings.filterwarnings('ignore')
6
7 mrf = yf.download("MRF.NS", start="2019-01-01",
8   end="2024-12-31")
```

```

8 # MRF.NS is the NSE ticker
9 close = mrf["Close"]
10
11 # Compute 30-day moving average
12 ma30 = close.rolling(window=30).mean()
13 # rolling(30) creates a sliding 30-day window, .mean()
    averages it
14
15 # Plot both
16 close.plot(label="MRF Close", alpha=0.5)
17 ma30.plot(label="30-Day MA", color="red")
18 plt.legend()
19 plt.title("MRF -- Price vs 30-Day Moving Average")
20 plt.show()

```

② Plain English

`rolling(window=30).mean()` slides a 30-day window across the series and computes the average at each step. The result is a smoothed version of the price — it removes day-to-day noise and reveals the underlying trend direction.

③ Common Mistake

Using `.average()` instead of `.mean()` — pandas uses `.mean()`. Also forgetting that the first 29 values of `ma30` will be `NaN` because there aren't enough prior points to fill the window — this is expected and normal.

④ Real World Link

When you see “MRF breaking above its 30-day moving average” in financial news, this is exactly the calculation they're referring to. Traders use moving averages as dynamic support/resistance levels.

M1.003 Fill in the Blank MEDIUM

Topic: Classical decomposition

Fill in the blanks to decompose the Gold futures series into its four components using **multiplicative** decomposition.

```

1 from statsmodels.tsa.seasonal import -----
2
3 gold = yf.download("GC=F", start="2019-01-01",
4     end="2024-12-31")
5 close = gold["Close"].dropna()
6
7 # Resample to monthly -- use last price of each month
8 monthly = close.resample("-----").last()
9
10 # Decompose -- multiplicative model
11 result = seasonal_decompose(monthly, model="-----",
    period=-----)

```

```

12 # Plot all four components
13 result._____.savefig("gold_decomposition.png")
14 plt.show()

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import matplotlib.pyplot as plt
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.seasonal import seasonal_decompose
7
8 gold = yf.download("GC=F", start="2019-01-01",
9                   end="2024-12-31")
9 close = gold["Close"].dropna()
10
11 # Resample to monthly -- use last price of each month
12 monthly = close.resample("ME").last() # "ME" = Month End
13                                         frequency
14
15 # Decompose -- multiplicative model
16 result = seasonal_decompose(monthly,
17                             model="multiplicative", period=12)
18 # period=12 for annual seasonality in monthly data
19
20 # Plot all four components
21 result.plot().savefig("gold_decomposition.png")
22 # .plot() returns a Figure object
23 plt.show()

```

② Plain English

`seasonal_decompose()` breaks a series into Trend + Seasonal + Residual components. We choose `model="multiplicative"` for Gold because the seasonal swings grow proportionally with price — if gold doubles, the seasonal variation also roughly doubles, which is the hallmark of multiplicative behavior.

③ Common Mistake

Using `period=30` on monthly data — the period should match the seasonal cycle length in your data's frequency units. Monthly data with annual seasonality → `period=12`. Daily data with weekly seasonality → `period=5` or `7`. Getting this wrong produces meaningless components.

④ Real World Link

Gold has genuine seasonal patterns — demand spikes around Diwali, the Indian wedding season (Oct–Dec), and Chinese New Year. The decomposition lets you strip

out this known seasonality to see what the “true” price trend looks like underneath.

M1.004 Fill in the Blank MEDIUM

Topic: Autocorrelation Function (ACF)

Fill in the blanks to plot the ACF of BTC daily returns and interpret the output.

```

1 from statsmodels.graphics.tsaplots import -----
2
3 btc = yf.download("BTC-USD", start="2019-01-01",
4                   end="2024-12-31")
5 close = btc["Close"]
6
7 # Compute daily log returns
8 returns = np.-----(close).diff().-----() # log returns,
9 drop NaN
10
11 # Plot ACF with 40 lags
12 fig, ax = plt.subplots(figsize=(12, 4))
13 -----(returns, lags=-----, ax=ax, alpha=0.05)
14 ax.set_title("ACF of BTC Daily Log Returns")
15 plt.show()

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import warnings
6 warnings.filterwarnings('ignore')
7 from statsmodels.graphics.tsaplots import plot_acf
8
9 btc = yf.download("BTC-USD", start="2019-01-01",
10                  end="2024-12-31")
11 close = btc["Close"]
12
13 # Compute daily log returns
14 returns = np.log(close).diff().dropna()
15 # log() computes natural log, diff() gives first
16 # difference,
17 # dropna() removes the first NaN
18
19 # Plot ACF with 40 lags
20 fig, ax = plt.subplots(figsize=(12, 4))
21 plot_acf(returns, lags=40, ax=ax, alpha=0.05)

```

```

20 # alpha=0.05 draws 95% confidence bands
21 ax.set_title("ACF of BTC Daily Log Returns")
22 plt.show()

```

② Plain English

The ACF plot shows how correlated today's return is with returns 1, 2, 3... days ago. Bars inside the blue shaded band are statistically insignificant — they're just noise. If most bars for BTC returns fall inside the band, that suggests returns are close to a white noise process (hard to predict).

③ Common Mistake

Plotting the ACF on the **price level** instead of **returns** — price levels are almost always non-stationary and will show a slowly decaying ACF that looks dramatic but is meaningless. Always difference or take log-returns before interpreting the ACF.

④ Real World Link

If BTC returns show significant ACF at lag 1, a quant trader could theoretically build a simple momentum or mean-reversion strategy. The ACF is the first checkpoint before building any predictive model.

M1.005 *Fill in the Blank* **HARD**

Topic: Ljung-Box Portmanteau Test

Fill in the blanks to run the **Ljung-Box test** on MRF returns and correctly interpret the p-value.

```

1 from statsmodels.stats.diagnostic import _____
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
4                 end="2024-12-31")
5 returns = np.log(mrf["Close"]).diff().dropna()
6
7 # Run Ljung-Box test for lags 1 to 20
8 lb_result = acorr_ljungbox(returns, lags=_____,
9                           return_df=_____)
10
11 print(lb_result)
12
13 # How many lags have p-value < 0.05?
14 significant = lb_result[lb_result["_____"] < 0.05]
15 print(f"\nLags with significant autocorrelation: {_____}")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf

```

```

3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.stats.diagnostic import acorr_ljungbox
7
8 mrf = yf.download("MRF.NS", start="2019-01-01",
9                  end="2024-12-31")
10 returns = np.log(mrf["Close"]).diff().dropna()
11
12 # Run Ljung-Box test for lags 1 to 20
13 lb_result = acorr_ljungbox(returns, lags=20,
14                           return_df=True)
15 # return_df=True gives a clean DataFrame output
16
17 print(lb_result)
18
19 # How many lags have p-value < 0.05?
20 significant = lb_result[lb_result["lb_pvalue"] < 0.05]
21 # "lb_pvalue" is the column name in the output DataFrame
22 print(f"\nLags with significant autocorrelation:
23       {len(significant)}")

```

② Plain English

The Ljung-Box test checks whether the autocorrelations up to lag m are **jointly** zero — i.e., is this series white noise? A p-value below 0.05 at any lag means we reject the null of white noise — there’s statistically significant structure left to model at that lag.

③ Common Mistake

Confusing the **null hypothesis** — the null is that the series IS white noise (no autocorrelation). So a small p-value is bad news for the “no structure” claim, meaning your series is NOT white noise and has patterns worth modeling. Students flip this constantly.

④ Real World Link

If MRF returns fail the Ljung-Box test ($p < 0.05$), it means past returns have predictive power over future returns — a direct challenge to the Efficient Market Hypothesis. This is why quants run this test before building trading strategies.

M1.006 Fill in the Blank **HARD**

Topic: Box-Cox transformation to achieve Gaussian process

Fill in the blanks to apply a **Box-Cox transformation** to Gold prices, then verify normality before and after.

```

1 from scipy.stats import _____, shapiro
2
3 gold = yf.download("GC=F", start="2019-01-01",
4                  end="2024-12-31")

```



```

4 close = gold["Close"].dropna().values # convert to numpy
   array
5
6 # Apply Box-Cox transformation
7 transformed, optimal_lambda = boxcox(_____)
8 print(f"Optimal lambda: {optimal_lambda:.4f}")
9
10 # Test normality BEFORE transformation
11 stat_before, p_before = shapiro(close[:_____])
12 print(f"Shapiro p-value BEFORE: {p_before:.4f}")
13
14 # Test normality AFTER transformation
15 stat_after, p_after = _____(transformed[:500])
16 print(f"Shapiro p-value AFTER: {p_after:.4f}")
17
18 # Interpret
19 if p_after _____ 0.05:
20     print("Transformation successful -- cannot reject
           normality")
21 else:
22     print("Still not Gaussian -- try log transform or
           differencing")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from scipy.stats import boxcox, shapiro
7
8 gold = yf.download("GC=F", start="2019-01-01",
9                   end="2024-12-31")
10 close = gold["Close"].dropna().values # convert to numpy
    array
11
12 # Apply Box-Cox transformation
13 transformed, optimal_lambda = boxcox(close)
14 # close must be strictly positive (Gold prices always are)
15 print(f"Optimal lambda: {optimal_lambda:.4f}")
16
17 # Test normality BEFORE transformation
18 stat_before, p_before = shapiro(close[:500])
19 # shapiro works on max 5000 samples; we use 500 to be safe
20 print(f"Shapiro p-value BEFORE: {p_before:.4f}")

```

```

20
21 # Test normality AFTER transformation
22 stat_after, p_after = shapiro(transformed[:500])
23 print(f"Shapiro p-value AFTER: {p_after:.4f}")
24
25 # Interpret
26 if p_after > 0.05:
27     # > 0.05 means we FAIL to reject the null of normality
28     print("Transformation successful -- cannot reject
29           normality")
30 else:
31     print("Still not Gaussian -- try log transform or
32           differencing")

```

② Plain English

Box-Cox finds the power transformation λ that makes your data most Gaussian. When $\lambda \approx 0$, it's essentially a log transform. When $\lambda = 1$, no transformation is needed. The Shapiro-Wilk test then confirms whether the transformed series is close enough to normal.

③ Common Mistake

Forgetting that Box-Cox **requires strictly positive values**. If your series has zeros or negatives (like returns), Box-Cox will crash. Also, $p > 0.05$ means we *fail to reject* normality — students confuse this direction.

④ Real World Link

Many ARIMA-family models assume Gaussian residuals. If raw Gold prices are skewed (heavy right tail from price spikes), Box-Cox transformation is a preprocessing step that makes model residuals better-behaved, improving forecast intervals.

Section B

Find the Error

M1.007 Find the Error **EASY**

Topic: Moving average — window size

This code is supposed to plot a **7-day** moving average of BTC prices. It runs without crashing but produces a **wrong result**. Find and fix the error.

```

1 btc = yf.download("BTC-USD", start="2019-01-01",
2   end="2024-12-31")
3 close = btc["Close"]
4 ma7 = close.rolling(window=7).sum()    # ERROR IS HERE
5
6 close.plot(label="BTC Close", alpha=0.4)
7 ma7.plot(label="7-Day MA", color="orange")
8 plt.legend()

```

```

9 plt.title("BTC -- 7-Day Moving Average")
10 plt.show()

```

What is wrong? What should it be?

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import matplotlib.pyplot as plt
4 import warnings
5 warnings.filterwarnings('ignore')
6
7 btc = yf.download("BTC-USD", start="2019-01-01",
8                 end="2024-12-31")
9 close = btc["Close"]
10
11 ma7 = close.rolling(window=7).mean()
12 # FIX: .sum() was used instead of .mean()
13 # Moving average = arithmetic mean, not sum
14
15 close.plot(label="BTC Close", alpha=0.4)
16 ma7.plot(label="7-Day MA", color="orange")
17 plt.legend()
18 plt.title("BTC -- 7-Day Moving Average")
19 plt.show()

```

② Plain English

A moving average is defined as the **mean** of the window, not the sum. Using `.sum()` produces values $7\times$ larger than prices — the “smoothed” line would appear far above the actual price series.

③ Common Mistake

This is a silent bug — the code runs without error, but the output is numerically wrong. Always sanity-check your MA values against the raw price: a 7-day MA of BTC should be roughly the same order of magnitude as BTC’s price, not $7\times$ larger.

④ Real World Link

A 7-day MA on BTC is used to smooth out weekend volatility effects. If you’re computing it with `.sum()` instead of `.mean()`, every trading signal derived from it would be garbage.

M1.008 Find the Error **EASY**

Topic: ACF import

This code crashes immediately. Find the error.

```

1 from statsmodels.graphics import plot_acf    # student
   wrote this
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
   end="2024-12-31")
4 returns = np.log(mrf["Close"]).diff().dropna()
5
6 plot_acf(returns, lags=30)
7 plt.show()

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import warnings
6 warnings.filterwarnings('ignore')
7
8 from statsmodels.graphics.tsaplots import plot_acf
9 # FIX: must import from .tsaplots submodule
10
11 mrf = yf.download("MRF.NS", start="2019-01-01",
   end="2024-12-31")
12 returns = np.log(mrf["Close"]).diff().dropna()
13
14 plot_acf(returns, lags=30)
15 plt.show()

```

② Plain English

`plot_acf` lives inside `statsmodels.graphics.tsaplots` — the `.tsaplots` part is required. Importing from `statsmodels.graphics` directly fails because `plot_acf` is not exported at that level.

③ Common Mistake

Guessing import paths. The error message `ImportError: cannot import name 'plot_acf' from 'statsmodels.graphics'` tells you exactly what failed — read error messages carefully, they're your first debugging tool.

④ Real World Link

Import errors are the #1 reason students lose marks in practicals. Knowing the exact import path by heart for `plot_acf`, `seasonal_decompose`, and `acorr_ljungbox` saves you precious minutes in a time-constrained exam.

M1.009 Find the Error MEDIUM

Topic: Seasonal decomposition — period mismatch

This code runs but produces a **meaningless decomposition**. Find the conceptual error and fix it.

```

1 gold = yf.download("GC=F", start="2019-01-01",
2   end="2024-12-31")
3
4 # Resample to monthly
5 monthly = close.resample("ME").last()
6
7 from statsmodels.tsa.seasonal import seasonal_decompose
8 result = seasonal_decompose(monthly, model="additive",
9   period=365) # ERROR
10
11 result.plot()
12 plt.show()

```

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import matplotlib.pyplot as plt
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.seasonal import seasonal_decompose
7
8 gold = yf.download("GC=F", start="2019-01-01",
9   end="2024-12-31")
10 close = gold["Close"].dropna()
11
12 monthly = close.resample("ME").last()
13
14 # FIX: period must match the frequency of your DATA, not
15   the calendar
16 # Monthly data -> annual seasonality = 12 months per cycle
17 result = seasonal_decompose(monthly, model="additive",
18   period=12)
19
20 result.plot()
21 plt.show()

```

② Plain English

The **period** parameter tells the decomposition how many **data points** make one seasonal cycle. Since **monthly** has one observation per month and we expect annual seasonality, the period is 12. Using **period=365** would tell the model to look for a

cycle that repeats every 365 months — about 30 years — which is nonsensical.

③ Common Mistake

Confusing calendar days with data frequency. Monthly data with annual seasonality → `period=12`. Daily data with weekly seasonality → `period=5` or `7`. Always ask: “how many *rows* of my data make one complete seasonal cycle?”

④ Real World Link

Gold’s genuine seasonal cycle is annual — 12 months. Using `period=365` on monthly data would attempt to find a 30-year cycle in 5 years of data, which is statistically impossible.

M1.010 Find the Error MEDIUM

Topic: Log returns — order of operations

This code runs without error but computes the **wrong thing**. Find the logical error.

```
1 btc = yf.download("BTC-USD", start="2019-01-01",
2   end="2024-12-31")
3   close = btc["Close"]
4   # Compute log returns -- WRONG
5   log_returns = np.log(close.diff()) # student's version
6
7   print(log_returns.describe())
8   print(f"Any negative values: {(log_returns < 0).any()}")
```

Solution

① Correct Code

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6
7 btc = yf.download("BTC-USD", start="2019-01-01",
8   end="2024-12-31")
9   close = btc["Close"]
10
11 # FIX: log returns = difference of logs, NOT log of
12   differences
13   log_returns = np.log(close).diff().dropna()
14 # Correct formula:  $r_t = \log(P_t) - \log(P_{t-1}) = \log(P_t / P_{t-1})$ 
15
16 print(log_returns.describe())
17 print(f"Any negative values: {(log_returns < 0).any()}")
```

② Plain English

Log returns are computed as $\log(P_t) - \log(P_{t-1}) = \log(P_t/P_{t-1})$. The student computed $\log(P_t - P_{t-1}) = \log$ of the price *change*, which breaks whenever the price change is negative (you can't take log of a negative number → produces NaN).

③ Common Mistake

This is one of the most common errors in quantitative finance. `np.log(close.diff())` will produce NaN wherever the price fell. The correct formula applies `np.log()` first, then `.diff()`. The difference matters: approximately 50% of BTC daily changes are negative, so this bug corrupts half your data.

④ Real World Link

Log returns are preferred over simple returns in finance because they're additive over time and approximately symmetric around zero. Every GARCH and ARIMA model you build on BTC will expect log returns as input — getting this formula right from day one is non-negotiable.

M1.011 Find the Error **HARD**

Topic: Exponential smoothing — alpha range

This code crashes. There are **two errors** — find both.

```
1 from statsmodels.tsa.holtwinters import SimpleExpSmoothing
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
4     end="2024-12-31")
5 close = mrf["Close"].dropna()
6
7 model = SimpleExpSmoothing(close)
8 fit = model.fit(smoothing_level=1.5, optimized=True)  #
9     ERROR 1
10
11 forecast = fit.forecast(steps="30")  # ERROR 2
12
13 plt.plot(close[-100:], label="Actual")
14 plt.plot(forecast, label="Forecast")
15 plt.legend()
16 plt.show()
```

Solution

① Correct Code

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import matplotlib.pyplot as plt
```

```

4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.holtwinters import SimpleExpSmoothing
7
8 mrf = yf.download("MRF.NS", start="2019-01-01",
9                  end="2024-12-31")
10 close = mrf["Close"].dropna()
11
12 model = SimpleExpSmoothing(close)
13 # FIX 1: smoothing_level (alpha) must be in (0, 1). 1.5 is
14 #        invalid.
15 # FIX 2: When optimized=True, do NOT manually set
16 #        smoothing_level
17 fit = model.fit(optimized=True) # let statsmodels find
18 #        optimal alpha
19
20 # FIX 3: steps must be an INTEGER, not a string
21 forecast = fit.forecast(steps=30)
22
23 plt.plot(close[-100:].values, label="Actual")
24 plt.plot(range(100, 130), forecast.values,
25          label="Forecast")
26 plt.legend()
27 plt.title("MRF -- SES Forecast")
28 plt.show()

```

② Plain English

Two bugs here. First: the smoothing parameter α must be strictly between 0 and 1 — it's a weighted average coefficient, so values ≥ 1 or ≤ 0 make no mathematical sense. Second: `steps` expects an integer, not a string.

③ Common Mistake

Setting `smoothing_level` and `optimized=True` simultaneously creates a conflict — if you're telling the model to optimize, why are you also manually setting the level? Either set `optimized=False` and provide your own alpha, or set `optimized=True` and let the model choose. Using both is contradictory.

④ Real World Link

For MRF, an optimally chosen α close to 1 means the model heavily weights recent prices (reacts fast to news like raw rubber price changes). An α close to 0 means it weights history more heavily (sluggish response). The optimizer chooses whichever minimizes forecast error on historical data.

M1.012 Find the Error **HARD**

Topic: Portmanteau test — interpretation logic

The code runs and prints results, but the **interpretation logic is inverted**. Find and fix the error.


```

1 from statsmodels.stats.diagnostic import acorr_ljungbox
2
3 gold = yf.download("GC=F", start="2019-01-01",
4                   end="2024-12-31")
5 returns = np.log(gold["Close"]).diff().dropna()
6
7 lb = acorr_ljungbox(returns, lags=10, return_df=True)
8
9 for lag, row in lb.iterrows():
10     if row["lb_pvalue"] < 0.05:
11         print(f"Lag {lag}: White noise confirmed")      #
12         ERROR: wrong
13     else:
14         print(f"Lag {lag}: Autocorrelation detected")    #
15         ERROR: wrong

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.stats.diagnostic import acorr_ljungbox
7
8 gold = yf.download("GC=F", start="2019-01-01",
9                   end="2024-12-31")
10 returns = np.log(gold["Close"]).diff().dropna()
11
12 lb = acorr_ljungbox(returns, lags=10, return_df=True)
13
14 # FIX: The null hypothesis of Ljung-Box IS white noise (no
15 # autocorrelation)
16 # p < 0.05 -> REJECT null -> autocorrelation EXISTS (NOT
17 # white noise)
18 # p >= 0.05 -> FAIL to reject null -> consistent with
19 # white noise
20
21 for lag, row in lb.iterrows():
22     if row["lb_pvalue"] < 0.05:
23         print(f"Lag {lag}: Autocorrelation detected -- NOT
24               white noise")
25     else:
26         print(f"Lag {lag}: Consistent with white noise")

```

② Plain English

The Ljung-Box test's null hypothesis (H_0) is that the series is white noise — no autocorrelation up to lag m . A small p-value (< 0.05) means we reject H_0 , meaning the series is **not** white noise and has significant autocorrelation. The original code had the conclusions perfectly backwards.

③ Common Mistake

Null hypothesis confusion is extremely common with diagnostic tests. Always write out H_0 and H_1 before interpreting any test. For Ljung-Box: H_0 = white noise (no autocorrelation), H_1 = not white noise (autocorrelation exists). Small $p \rightarrow$ reject $H_0 \rightarrow H_1$ is supported.

④ Real World Link

If Gold returns fail the Ljung-Box test (small $p \rightarrow$ not white noise), it means there's predictable structure — perhaps GARCH effects (volatility clustering) even if mean returns look random. This is the gateway finding that motivates GARCH modeling in Module 4.

Section C

Get the Order Right

M1.013 *Get the Order Right* **EASY**

Topic: Steps to compute and plot ACF

The following **6 steps** for plotting the ACF of MRF returns are scrambled. Write the correct order by numbering them 1–6.

- (A) `plot.acf(returns, lags=40, ax=ax)`
- (B) `mrf = yf.download("MRF.NS", start="2019-01-01", end="2024-12-31")`
- (C) `from statsmodels.graphics.tsaplots import plot_acf`
- (D) `plt.show()`
- (E) `returns = np.log(mrf["Close"]).diff().dropna()`
- (F) `fig, ax = plt.subplots(figsize=(10, 4))`

Solution

① **Correct Order:** C $\rightarrow$ B $\rightarrow$ E $\rightarrow$ F $\rightarrow$ A $\rightarrow$ D

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import warnings
6 warnings.filterwarnings('ignore')
7
```

```

8 # Step C: Import the function you need
9 from statsmodels.graphics.tsaplots import plot_acf
10
11 # Step B: Download the data
12 mrf = yf.download("MRF.NS", start="2019-01-01",
13                  end="2024-12-31")
14
15 # Step E: Compute log returns
16 returns = np.log(mrf["Close"]).diff().dropna()
17
18 # Step F: Create figure and axes
19 fig, ax = plt.subplots(figsize=(10, 4))
20
21 # Step A: Plot ACF
22 plot_acf(returns, lags=40, ax=ax)
23
24 # Step D: Display
25 plt.show()

```

② Plain English

The logical flow is always: import → fetch data → transform data → set up plot → draw plot → display. Skipping or reordering any step causes either an ImportError, a NameError, or a blank screen.

③ Common Mistake

Calling `plt.show()` before the plotting function — this creates an empty window. Always call `plt.show()` as the last step after all drawing commands have been issued.

④ Real World Link

In a real analysis pipeline, this exact sequence (import → fetch → transform → visualize) is repeated hundreds of times. Internalizing the order means your workflow becomes automatic — you stop thinking about Python and start thinking about what the ACF is telling you.

M1.014 Get the Order Right **EASY**

Topic: Seasonal decomposition pipeline

Reorder these **5 steps** for correctly decomposing the Gold price series.

- (A) `result.plot(); plt.show()`
- (B) `monthly = close.resample("ME").last()`
- (C) `close = gold["Close"].dropna()`
- (D) `result = seasonal_decompose(monthly, model="multiplicative", period=12)`
- (E) `gold = yf.download("GC=F", start="2019-01-01", end="2024-12-31")`

Solution**① Correct Order: E → C → B → D → A**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import matplotlib.pyplot as plt
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.seasonal import seasonal_decompose
7
8 # Step E: Download raw data
9 gold = yf.download("GC=F", start="2019-01-01",
10                    end="2024-12-31")
11
12 # Step C: Extract and clean closing prices
13 close = gold["Close"].dropna()
14
15 # Step B: Resample from daily to monthly frequency
16 monthly = close.resample("ME").last()
17
18 # Step D: Apply seasonal decomposition
19 result = seasonal_decompose(monthly,
20                             model="multiplicative", period=12)
21
22 # Step A: Visualize all four components
23 result.plot()
24 plt.show()

```

② Plain English

Data must exist before you can select from it (E before C), must be in one column before you resample it (C before B), must be in correct frequency before decomposing (B before D), and must be decomposed before plotting (D before A). Each step depends on the output of the previous one.

③ Common Mistake

Trying to apply `resample()` on the full OHLCV DataFrame instead of on the extracted `close` Series — extracting the column first (Step C) keeps things clean and explicit.

④ Real World Link

This pipeline — download → clean → resample → decompose → visualize — is standard in any financial time series analysis. Mastering the order means you can apply it to any asset: BTC weekly cycles, MRF quarterly earnings patterns, Gold monsoon-season demand.

M1.015 *Get the Order Right* **MEDIUM**

Topic: Box-Cox transformation and normality verification

Reorder these **7 steps** to correctly transform Gold prices and test for normality

before and after.

- (A) `stat_after, p_after = shapiro(transformed[:500])`
- (B) `from scipy.stats import boxcox, shapiro`
- (C) `print(f"Before: p={p_before:.4f}, After: p={p_after:.4f}")`
- (D) `close = gold["Close"].dropna().values`
- (E) `stat_before, p_before = shapiro(close[:500])`
- (F) `gold = yf.download("GC=F", start="2019-01-01", end="2024-12-31")`
- (G) `transformed, lam = boxcox(close)`

Solution

① Correct Order: B → F → D → E → G → A → C

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import warnings
4 warnings.filterwarnings('ignore')
5
6 # Step B: Import required functions
7 from scipy.stats import boxcox, shapiro
8
9 # Step F: Download data
10 gold = yf.download("GC=F", start="2019-01-01",
11                    end="2024-12-31")
12
13 # Step D: Extract closing prices as numpy array
14 close = gold["Close"].dropna().values
15
16 # Step E: Test normality BEFORE transformation
17 stat_before, p_before = shapiro(close[:500])
18
19 # Step G: Apply Box-Cox transformation
20 transformed, lam = boxcox(close)
21
22 # Step A: Test normality AFTER transformation
23 stat_after, p_after = shapiro(transformed[:500])
24
25 # Step C: Print comparison
26 print(f"Before: p={p_before:.4f}, After: p={p_after:.4f}")
27 print(f"Optimal lambda: {lam:.4f}")

```

② Plain English

Import comes first (you need the tools before using them), data download comes next (you need data before testing it), and the “after” test needs `transformed` to exist first. Variables must be created before they can be referenced — Python executes top to bottom.

③ Common Mistake

Running Step A before Step G — trying to test `transformed` before creating it gives a `NameError: name 'transformed' is not defined`.

④ Real World Link

In practice, you always benchmark normality before and after a transformation to quantify the improvement. If p goes from 0.001 to 0.12 after Box-Cox, that's meaningful progress toward satisfying the Gaussian assumption required by ARIMA models.

M1.016 *Get the Order Right* MEDIUM

Topic: Exponential smoothing with Holt's method (trend)

Reorder these **6 steps** to fit Holt's double exponential smoothing on BTC and forecast 90 days ahead.

- (A) `forecast = fit.forecast(steps=90)`
- (B) `from statsmodels.tsa.holtwinters import Holt`
- (C) `fit = model.fit(optimized=True)`
- (D) `close = btc["Close"].dropna()`
- (E) `model = Holt(close)`
- (F) `btc = yf.download("BTC-USD", start="2019-01-01", end="2024-12-31")`

Solution

① **Correct Order:** B → F → D → E → C → A

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import matplotlib.pyplot as plt
4 import warnings
5 warnings.filterwarnings('ignore')
6
7 # Step B: Import Holt's method
8 from statsmodels.tsa.holtwinters import Holt
9
10 # Step F: Download BTC data
11 btc = yf.download("BTC-USD", start="2019-01-01",
12                  end="2024-12-31")
13
14 # Step D: Extract closing prices
15 close = btc["Close"].dropna()
16
17 # Step E: Instantiate the model with your data
18 model = Holt(close)

```

```

19 # Step C: Fit the model
20 fit = model.fit(optimized=True)
21
22 # Step A: Generate 90-day forecast
23 forecast = fit.forecast(steps=90)

```

② Plain English

The pattern for all statsmodels models is: import → data → instantiate → fit → forecast. This is the same pattern you'll use for ARIMA, SARIMA, GARCH — memorize it now and you'll recognize it across all modules.

③ Common Mistake

Calling `fit.forecast()` before `model.fit()` — you cannot forecast from an unfitted model. The `fit` object is produced by `.fit()` and contains the estimated parameters needed to generate forecasts.

④ Real World Link

Holt's method captures BTC's long-run upward trend while smoothing short-term noise. The 90-day forecast will show a straight-line trend projection — useful for rough planning but will obviously miss the dramatic reversals that make BTC famous.

Section D

Match the Following

M1.017 Match the Following **EASY**

Topic: Time series components

Match each **component** (Column A) to its **description** (Column B).

#	Column A — Component	#	Column B — Description
1	Trend	(a)	Repeating pattern at fixed calendar intervals
2	Seasonality	(b)	Long-run upward or downward movement in the mean
3	Periodicity	(c)	Unpredictable, random fluctuations
4	Irregular/Residual	(d)	Cycles not tied to fixed calendar intervals

Solution

① Correct Matches

Component	Match
1. Trend	(b) Long-run upward or downward movement in the mean
2. Seasonality	(a) Repeating pattern at fixed calendar intervals
3. Periodicity	(d) Cycles not tied to fixed calendar intervals
4. Irregular/Residual	(c) Unpredictable, random fluctuations

② Plain English

A time series is the sum (or product) of these four components. Trend is the big story (BTC generally went up 2019–2021). Seasonality is predictable repetition (Gold demand every Diwali). Periodicity is irregular cycling (business cycles). The residual is everything left over after removing the other three.

③ Common Mistake

Confusing seasonality and periodicity. Seasonality has a **fixed, known period** — exactly 12 months for annual patterns. Periodicity has **variable cycle lengths** — business cycles run anywhere from 2 to 10 years with no fixed period. The calendar-anchoring is the key difference.

④ Real World Link

For MRF, trend = gradual share price appreciation over years. Seasonality = Q1 sales bump (summer tyre replacement season). Periodicity = multi-year auto industry cycles. Irregular = the COVID crash in March 2020. Every news headline you read maps to one of these components.

M1.018 Match the Following **EASY**

Topic: Python functions and their purpose

Match each **function** (Column A) to what it **does** (Column B).

#	Column A — Function	#	Column B — What it does
1	<code>rolling(window=7).mean()</code>	(a)	Tests whether autocorrelations are jointly zero
2	<code>seasonal_decompose()</code>	(b)	Computes a simple moving average
3	<code>plot_acf()</code>	(c)	Splits a series into trend, seasonal, residual
4	<code>acorr_ljungbox()</code>	(d)	Visualizes correlation between a series and its own lags

Solution

① Correct Matches

Function	Match
1. <code>rolling(window=7).mean()</code>	(b) Computes a simple moving average
2. <code>seasonal_decompose()</code>	(c) Splits a series into trend, seasonal, residual
3. <code>plot_acf()</code>	(d) Visualizes correlation between a series and its own lags
4. <code>acorr_ljungbox()</code>	(a) Tests whether autocorrelations are jointly zero

② Plain English

These four functions cover the entire Module 1 toolkit — smoothing, decomposition, correlation visualization, and statistical testing. You'll use all four in every exploratory analysis of any financial time series.

③ Common Mistake

Thinking `plot_acf()` and `acorr_ljungbox()` test the same thing — they don't. `plot_acf()` *visualizes* individual lag correlations without a formal test. `acorr_ljungbox()` *tests* whether the group of autocorrelations up to lag m is jointly zero. One is exploratory, one is confirmatory.

④ Real World Link

In a real financial analysis report, you'd use `plot_acf()` in your exploratory section ("here's what we saw") and `acorr_ljungbox()` in your results section ("here's the statistical evidence"). Both serve different audiences — one visual, one quantitative.

M1.019 Match the Following **MEDIUM**

Topic: Smoothing parameters and their effect

Match each **alpha value** (Column A) to its **smoothing behavior** (Column B).

#	Column A — Alpha (α)	#	Column B — Behavior
1	$\alpha = 0.05$	(a)	Almost no smoothing — tracks raw data closely
2	$\alpha = 0.95$	(b)	Heavy smoothing — very slow to respond to new data
3	$\alpha = 0.5$	(c)	Balanced — equal weight to recent and historical data
4	$\alpha = 1.0$	(d)	Invalid for standard SES — boundary condition

Solution

① Correct Matches

Alpha	Match
1. $\alpha = 0.05$	(b) Heavy smoothing — very slow to respond to new data
2. $\alpha = 0.95$	(a) Almost no smoothing — output tracks raw data closely
3. $\alpha = 0.5$	(c) Balanced — equal weight to recent and historical data
4. $\alpha = 1.0$	(d) Invalid for standard SES — boundary condition

② Plain English

In exponential smoothing, the formula is $S_t = \alpha \cdot X_t + (1 - \alpha) \cdot S_{t-1}$. Small $\alpha \rightarrow$ large weight on history $\rightarrow$ smoother but lagging output. Large $\alpha \rightarrow$ large weight on the latest observation $\rightarrow$ reactive but noisy output.

③ Common Mistake

Thinking larger α = more smoothing. It's the opposite. Large α means the model “forgets” old data fast and reacts strongly to the latest point — which is LESS smooth, not more.

④ Real World Link

For Gold (slow-moving, trend-dominated), you'd prefer small α to capture the gradual trend without overreacting to daily noise. For BTC (volatile, fast-moving), traders might prefer large α to stay responsive to rapid price changes. The optimal α for each asset is different.

M1.020 Match the Following **MEDIUM**

Topic: ACF pattern $\rightarrow$ process type

Match the **ACF pattern description** (Column A) to the **process it suggests** (Column B).

#	Column A — ACF Pattern	#	Column B — Process
1	Decays slowly, stays positive for many lags	(a)	White noise (no structure)
2	All spikes within confidence bands	(b)	Non-stationary series (e.g., price levels)
3	Single spike at lag 1, then cuts off	(c)	MA(1) process
4	Alternating positive/negative spikes, decaying	(d)	AR(1) process with negative coefficient

Solution

① Correct Matches

ACF Pattern	Match
1. Slow positive decay	(b) Non-stationary series (e.g., price levels)
2. All within bands	(a) White noise (no structure)
3. Spike at lag 1, cuts off	(c) MA(1) process
4. Alternating decaying spikes	(d) AR(1) with negative coefficient

② Plain English

Reading ACF patterns is a diagnostic skill. Slow decay $\rightarrow$ non-stationary, need to difference. All inside bands $\rightarrow$ no structure to model. Single lag-1 spike $\rightarrow$ MA(1). Alternating decay $\rightarrow$ AR with negative coefficient (the process overshoots

and corrects). You'll use this skill every time you identify an ARIMA model in Modules 2 and 3.

③ Common Mistake

Mistaking a slowly decaying ACF (non-stationarity) for a high-order MA process. The slow, smooth, persistent decay is the signature of a unit root, not a long-memory MA. The fix is differencing, not adding MA terms.

④ Real World Link

BTC's price level ACF will look like pattern 1 (slow positive decay) — classic non-stationary behavior. BTC's log returns ACF often looks like pattern 2 (mostly within bands) — which is why simple ARIMA models struggle to beat random walk forecasts for crypto prices.

Section E

Multiple Choice

M1.021 Multiple Choice **EASY**

Topic: Moving average — window effect

Which statement best describes the effect of **increasing the window size** of a moving average?

- (A) The smoothed series becomes noisier and more reactive to recent data
- (B) The smoothed series lags further behind and removes more short-term variation
- (C) The smoothed series becomes identical to the original series
- (D) The number of NaN values at the start decreases

Solution

① Correct Answer: (B)

② Plain English

A larger window means each average is computed over more data points, which makes the output smoother but slower to respond to changes. A 200-day MA of BTC will show a gentle, gradual curve while a 7-day MA will wiggle with every market move. Larger window = more smoothing = more lag.

③ Common Mistake

Choosing (D) — actually the opposite is true. A window of size k creates $k - 1$ leading NaN values. Bigger window $\rightarrow$ more NaN values at the start, not fewer.

④ Real World Link

Technical analysts use both the 50-day and 200-day moving averages for BTC simultaneously. When the 50-day crosses above the 200-day (Golden Cross), it's a bullish signal. When it crosses below (Death Cross), it's bearish.

M1.022 Multiple Choice **EASY****Topic:** Additive vs Multiplicative decomposition

MRF's share price seasonal swings are roughly **2,000** regardless of whether the stock is at 50,000 or 90,000. Which decomposition model is appropriate?

- (A) Multiplicative — because prices are always positive
- (B) Additive — because seasonal variation is constant in absolute terms
- (C) Multiplicative — because the series has a trend
- (D) Either model will give identical results

Solution**① Correct Answer: (B)****② Plain English**

If seasonal swings stay roughly constant in *absolute* (rupee) terms regardless of the price level, use additive decomposition: $Y = \text{Trend} + \text{Seasonal} + \text{Residual}$. If seasonal swings grow *proportionally* with the price level, use multiplicative: $Y = \text{Trend} \times \text{Seasonal} \times \text{Residual}$.

③ Common Mistake

Choosing (A) — positivity alone does not determine the model choice. Both decompositions work on positive series. The choice depends purely on whether the seasonal amplitude scales with the level of the series.

④ Real World Link

For Gold, the seasonal demand spike (Diwali buying) is proportionally similar — if gold doubles in price, the seasonal demand push also roughly doubles in rupee terms, making multiplicative more appropriate.

M1.023 Multiple Choice **MEDIUM****Topic:** Log returns — properties

Which of the following is **NOT** a reason to use log returns instead of simple percentage returns?

- (A) Log returns are additive over time: multi-period return = sum of single-period log returns
- (B) Log returns are approximately symmetric around zero for small changes
- (C) Log returns eliminate all autocorrelation in the series
- (D) Log returns approximate simple returns for small percentage changes

Solution**① Correct Answer: (C)****② Plain English**

Log returns have three well-known advantages: time-additivity (A), approximate symmetry (B), and equivalence to simple returns for small changes (D). But they

do **not** eliminate autocorrelation — whether autocorrelation exists depends on the underlying market dynamics, not the return calculation method.

③ Common Mistake

Believing that the log transformation “fixes” all statistical problems. Log returns are better-behaved than simple returns, but they don’t remove autocorrelation, heteroscedasticity, or heavy tails — which is why we still need GARCH models in Module 4 even after computing log returns.

④ Real World Link

BTC log returns from 2019–2024 will still show volatility clustering (large moves followed by large moves) even though they’re log-transformed. Log returns simply make the series more tractable to work with.

M1.024 Multiple Choice MEDIUM

Topic: ACF confidence bands

In a `plot.acf()` output with `alpha=0.05`, the blue shaded region represents:

- (A) The range of values the series itself takes
- (B) The 95% confidence interval — spikes outside indicate significant autocorrelation
- (C) The 5% trimmed mean of the ACF values
- (D) The region where the series is non-stationary

Solution

① Correct Answer: (B)

② Plain English

The blue bands in `plot.acf()` are confidence intervals at the significance level specified by `alpha`. With `alpha=0.05`, any spike outside the band is statistically significant at the 5% level — meaning we’re 95% confident that autocorrelation at that lag is not just random noise.

③ Common Mistake

Thinking that all spikes outside the band must be modeled. In practice, with 40 lags and `alpha=0.05`, you’d expect about 2 spurious significant spikes just by chance (5% of 40). Focus on spikes that are clearly outside the bands, especially at low lags, and look for consistent patterns rather than isolated spikes.

④ Real World Link

When you plot the ACF of BTC log returns and see most spikes inside the bands with only 1–2 marginal exceptions, that’s telling you BTC returns are close to white noise — which is consistent with market efficiency.

M1.025 Multiple Choice **HARD****Topic:** Ljung-Box test vs Box-Pierce testWhat is the key difference between the **Ljung-Box** test and the **Box-Pierce** test?

- (A) Ljung-Box tests for seasonality; Box-Pierce tests for trend
- (B) Ljung-Box is a finite-sample correction that performs better in small samples; Box-Pierce is the simpler original version
- (C) Box-Pierce uses the ACF; Ljung-Box uses the PACF
- (D) They are identical — different names for the same test

Solution**① Correct Answer: (B)****② Plain English**

Both tests check whether a group of autocorrelations are jointly zero. The Box-Pierce statistic is $n \cdot \sum \rho^2(k)$. The Ljung-Box statistic applies a finite-sample correction: $n(n+2) \cdot \sum [\rho^2(k)/(n-k)]$. For large samples they converge, but for small samples ($n < 100$), Ljung-Box is more accurately calibrated.

③ Common Mistake

Choosing (D) — while they test the same null hypothesis and often give similar results, they are mathematically different and can diverge meaningfully in small samples. Always report the Ljung-Box version in academic work.

④ Real World Link

With 5 years of daily financial data (≈ 1250 observations), both tests will give nearly identical results. But if you're working with monthly data (60 observations) for a specialized commodity, the difference between the two statistics could change your conclusion — use Ljung-Box to be safe.

M1.026 Multiple Choice **HARD****Topic:** Box-Cox lambda interpretationAfter running `boxcox(gold_close)`, the optimal lambda is returned as **0.12**. What does this imply?

- (A) Apply a square root transformation ($\lambda = 0.5$ is the closest standard value)
- (B) No transformation is needed — the series is already Gaussian
- (C) The optimal transformation is approximately a **log transform** ($\lambda \approx 0 \rightarrow \log$)
- (D) Apply a reciprocal transformation ($\lambda = -1$)

Solution**① Correct Answer: (C)****② Plain English**

The Box-Cox transformation formula is $(X^\lambda - 1)/\lambda$ for $\lambda \neq 0$, and $\log(X)$ when $\lambda = 0$. When the optimal λ is close to 0 (like 0.12), the Box-Cox transformation is

approximately equivalent to taking the natural log. For Gold prices with $\lambda \approx 0.12$, using `np.log(close)` is a reasonable and interpretable approximation.

③ Common Mistake

Choosing (A) — square root corresponds to $\lambda = 0.5$, which is far from 0.12. Standard reference values: $\lambda = 1$ (no transform), $\lambda = 0.5$ (square root), $\lambda = 0$ (log), $\lambda = -1$ (reciprocal).

④ Real World Link

The fact that Gold prices need a near-log transformation is consistent with financial theory — prices are multiplicative processes, and log-transforming them converts percentage changes into additive returns. Box-Cox is confirming what finance theory already knew: work in log-space.

Section F

Write from Scratch

M1.027 Write from Scratch **EASY**

Topic: Downloading and plotting raw data

Given this output description, write the complete code from scratch:

“A plot with two lines: BTC daily closing price (blue, semi-transparent) and its 50-day moving average (red, solid). Title: ‘BTC — Price and 50-Day Trend’. X-axis: Date. Y-axis: USD. Legend shown.”

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import matplotlib.pyplot as plt
4 import warnings
5 warnings.filterwarnings('ignore')
6
7 # Download BTC data
8 btc = yf.download("BTC-USD", start="2019-01-01",
9                 end="2024-12-31")
10 close = btc["Close"]
11
12 # Compute 50-day moving average
13 ma50 = close.rolling(window=50).mean()
14
15 # Create the plot
16 plt.figure(figsize=(14, 5))
17 plt.plot(close.index, close, color="blue", alpha=0.4,
18         label="BTC Close")

```

```

17 plt.plot(ma50.index, ma50, color="red", linewidth=2,
18          label="50-Day MA")
19
20 plt.title("BTC -- Price and 50-Day Trend")
21 plt.xlabel("Date")
22 plt.ylabel("USD")
23 plt.legend()
24 plt.tight_layout()
25 plt.show()

```

② Plain English

This is the fundamental chart in technical analysis. The raw price (blue) shows all volatility; the 50-day MA (red) shows the underlying direction. When BTC price crosses its MA from below, technicians call it a bullish signal.

③ Common Mistake

Forgetting `plt.tight_layout()` — without it, axis labels often get cut off in Colab. Also, many students use `close.plot()` shorthand, which works but gives less control over colors, linewidths, and labels.

④ Real World Link

Every crypto charting tool — TradingView, CoinGecko, Binance — shows this exact chart. When you read “BTC is trading above its 50-day MA,” you now know exactly how that line is calculated and what it means.

M1.028 Write from Scratch MEDIUM

Topic: Multi-window smoothing comparison

Write code that:

1. Downloads Gold futures (GC=F) from 2019
2. Computes 20-day, 50-day, and 200-day moving averages
3. Plots all four lines (raw + 3 MAs) on a single chart with distinct colors and a legend
4. Adds a vertical dashed line at 2020-03-16 (COVID crash day)

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import matplotlib.pyplot as plt
4 import pandas as pd
5 import warnings
6 warnings.filterwarnings('ignore')
7
8 gold = yf.download("GC=F", start="2019-01-01",
9                   end="2024-12-31")

```



```

9  close = gold["Close"]
10
11  ma20  = close.rolling(window=20).mean()    # short-term
      trend
12  ma50  = close.rolling(window=50).mean()    # medium-term
      trend
13  ma200 = close.rolling(window=200).mean()   # long-term trend
14
15  plt.figure(figsize=(16, 6))
16  plt.plot(close, color="gray", alpha=0.4, linewidth=1,
      label="Gold Close")
17  plt.plot(ma20, color="blue", linewidth=1.5,
      label="20-Day MA")
18  plt.plot(ma50, color="orange", linewidth=1.5,
      label="50-Day MA")
19  plt.plot(ma200, color="red", linewidth=2,
      label="200-Day MA")
20
21  plt.axvline(x=pd.Timestamp("2020-03-16"), color="black",
22              linestyle="--", linewidth=1.5, label="COVID
      Crash (16 Mar 2020)")
23
24  plt.title("Gold Futures -- Multi-Window Moving Average
      Comparison")
25  plt.xlabel("Date")
26  plt.ylabel("USD per oz")
27  plt.legend()
28  plt.tight_layout()
29  plt.show()

```

② Plain English

Comparing multiple MA windows on the same chart reveals how different “time-frames” of smoothing behave differently. The 20-day MA reacts quickly (good for traders); the 200-day MA is very stable (good for long-term investors). The vertical line anchors the visual around a known market event.

③ Common Mistake

Forgetting `pd.Timestamp()` when passing a date to `plt.axvline()` — always convert explicitly for reliability across matplotlib versions.

④ Real World Link

Notice how Gold initially dropped on the COVID crash (flight to cash) but then surged dramatically as central banks printed money. The 200-day MA helps you see the true trend change through the noise.

M1.029 Write from Scratch **MEDIUM****Topic:** ACF and PACF side-by-side

Write code that:

1. Downloads MRF data from 2019
2. Computes log returns
3. Plots ACF and PACF **side by side** in a single figure (1 row, 2 columns)
4. Uses 40 lags and 95% confidence bands for both
5. Adds a main title: “MRF Log Returns — ACF and PACF”

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import warnings
6 warnings.filterwarnings('ignore')
7 from statsmodels.graphics.tsaplots import plot_acf,
   plot_pacf
8
9 mrf = yf.download("MRF.NS", start="2019-01-01",
   end="2024-12-31")
10 close = mrf["Close"]
11 returns = np.log(close).diff().dropna()
12
13 fig, axes = plt.subplots(1, 2, figsize=(16, 5))
14
15 plot_acf(returns, lags=40, alpha=0.05, ax=axes[0])
16 axes[0].set_title("ACF -- MRF Log Returns")
17
18 plot_pacf(returns, lags=40, alpha=0.05, ax=axes[1])
19 axes[1].set_title("PACF -- MRF Log Returns")
20
21 fig.suptitle("MRF Log Returns -- ACF and PACF",
   fontsize=14, fontweight="bold")
22 plt.tight_layout()
23 plt.show()

```

② Plain English

Plotting ACF and PACF together is the standard model identification step for ARIMA. The ACF tells you about MA order (how many lags before it cuts off), and the PACF tells you about AR order. Reading them together is the foundation of Box-Jenkins model selection, which you'll master in Module 2.

③ Common Mistake

Forgetting to import `plot_pacf` separately from `plot_acf`. Also, using `plt.subplots(2, 1)` instead of `plt.subplots(1, 2)` creates a stacked layout (ver-

tical) rather than side-by-side (horizontal).

④ Real World Link

When analysts say “the PACF cuts off at lag 2, suggesting an AR(2) model,” this is exactly the plot they’re reading. For MRF, you might find that the PACF has a significant spike at lag 1 and cuts off — that’s the first clue that an AR(1) could describe the return dynamics.

M1.030 Write from Scratch **HARD**

Topic: Rolling correlation as adaptive smoothing

Write code that:

1. Downloads both BTC (BTC-USD) and Gold (GC=F) from 2019
2. Aligns them on the same dates (inner join)
3. Computes log returns for both
4. Computes the **60-day rolling correlation** between the two return series
5. Plots the rolling correlation over time with a horizontal dashed line at 0
6. Annotates the chart with: “Positive = BTC and Gold move together”

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import warnings
6 warnings.filterwarnings('ignore')
7
8 btc = yf.download("BTC-USD", start="2019-01-01",
9                 end="2024-12-31")["Close"]
10
11 gold = yf.download("GC=F", start="2019-01-01",
12                  end="2024-12-31")["Close"]
13
14 # Align on common dates using inner join
15 combined = btc.to_frame("BTC").join(gold.to_frame("Gold"),
16                                   how="inner")
17
18 # Log returns for both
19 returns = np.log(combined).diff().dropna()
20
21 # 60-day rolling correlation
22 roll_corr =
23     returns["BTC"].rolling(window=60).corr(returns["Gold"])
24
25 fig, ax = plt.subplots(figsize=(14, 5))

```

```

21 ax.plot(roll_corr.index, roll_corr, color="purple",
22         linewidth=1.5,
23         label="60-Day Rolling Correlation")
24 ax.axhline(y=0, color="black", linestyle="--", linewidth=1,
25           label="Zero Correlation")
26
27 ax.fill_between(roll_corr.index, roll_corr, 0,
28               where=(roll_corr > 0), alpha=0.2,
29               color="green")
30 ax.fill_between(roll_corr.index, roll_corr, 0,
31               where=(roll_corr < 0), alpha=0.2,
32               color="red")
33
34 ax.annotate("Positive = BTC and Gold move together",
35            xy=(roll_corr.index[200], 0.6), fontsize=10,
36            color="green")
37
38 ax.set_title("BTC vs Gold -- 60-Day Rolling Correlation of
39             Log Returns")
40 ax.set_xlabel("Date")
41 ax.set_ylabel("Correlation")
42 ax.legend()
43 plt.tight_layout()
44 plt.show()

```

② Plain English

Rolling correlation shows how the relationship between two assets **changes over time** — something a single static correlation coefficient hides. During the 2020 COVID crash, BTC and Gold briefly correlated strongly (both fell in the liquidity panic); during crypto bull markets, that relationship breaks down.

③ Common Mistake

Not aligning dates before computing correlation — BTC trades 365 days/year including weekends, while Gold futures don't trade on weekends. Misaligned indices produce NaN-filled correlations or wrong pairings. The inner join solves this by keeping only dates where both have data.

④ Real World Link

This rolling correlation chart is a key input for portfolio construction. When BTC-Gold correlation is low or negative, holding both provides genuine diversification. When it spikes positive (crisis periods), the diversification benefit disappears at exactly the wrong time — a phenomenon called “correlation breakdown in crises.”

M1.031 Write from Scratch **HARD**

Topic: Comprehensive exploratory analysis function

Write a reusable function `analyze_series(ticker, name, start="2019-01-01")` that:

1. Downloads the data and computes log returns
2. Produces a **2×2 figure**: (top-left) price with 50-day MA, (top-right) log returns, (bottom-left) ACF of returns, (bottom-right) return distribution histogram with a normal curve overlay
3. Runs Ljung-Box test (20 lags) and prints whether returns are white noise at 5% significance
4. Works for any of the three course tickers

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import scipy.stats as stats
7 import warnings
8 warnings.filterwarnings('ignore')
9 from statsmodels.graphics.tsaplots import plot_acf
10 from statsmodels.stats.diagnostic import acorr_ljungbox
11
12 def analyze_series(ticker, name, start="2019-01-01"):
13     data = yf.download(ticker, start=start,
14                        end="2024-12-31", progress=False)
15     close = data["Close"].dropna()
16     returns = np.log(close).diff().dropna()
17     ma50 = close.rolling(window=50).mean()
18
19     fig, axes = plt.subplots(2, 2, figsize=(16, 10))
20     fig.suptitle(f"{name} ({ticker}) -- Exploratory Time
21                 Series Analysis",
22                 fontsize=14, fontweight="bold")
23
24     # TOP LEFT: Price + 50-day MA
25     ax = axes[0, 0]
26     ax.plot(close, color="steelblue", alpha=0.5,
27            linewidth=1, label="Close Price")
28     ax.plot(ma50, color="red", linewidth=2, label="50-Day
29            MA")
30     ax.set_title("Price and 50-Day Moving Average")
31     ax.legend(fontsize=8)
32
33     # TOP RIGHT: Log Returns
34     ax = axes[0, 1]
35     ax.plot(returns, color="darkgreen", alpha=0.6,
36            linewidth=0.8)

```

```

32     ax.axhline(0, color="black", linewidth=0.8,
33               linestyle="--")
34     ax.set_title("Daily Log Returns")
35
36     # BOTTOM LEFT: ACF of Returns
37     ax = axes[1, 0]
38     plot_acf(returns, lags=40, alpha=0.05, ax=ax)
39     ax.set_title("ACF of Log Returns (40 lags)")
40
41     # BOTTOM RIGHT: Return Distribution
42     ax = axes[1, 1]
43     ax.hist(returns, bins=80, density=True,
44            color="purple", alpha=0.5,
45            label="Observed")
46     x = np.linspace(returns.min(), returns.max(), 300)
47     mu, sigma = returns.mean(), returns.std()
48     ax.plot(x, stats.norm.pdf(x, mu, sigma), color="red",
49            linewidth=2,
50            label="Normal Fit")
51     ax.set_title("Return Distribution vs Normal")
52     ax.legend(fontsize=8)
53
54     plt.tight_layout()
55     plt.show()
56
57     # Ljung-Box test
58     lb = acorr_ljungbox(returns, lags=20, return_df=True)
59     min_pval = lb["lb_pvalue"].min()
60     print(f"\n{'='*50}")
61     print(f"    Ljung-Box Test: {name} (20 lags)")
62     print(f"    Minimum p-value: {min_pval:.4f}")
63     if min_pval < 0.05:
64         print("    -> REJECT white noise -- significant
65               autocorrelation found")
66     else:
67         print("    -> FAIL TO REJECT white noise -- returns
68               appear random")
69     print(f"{'='*50}\n")
70
71     # Usage
72     analyze_series("BTC-USD", "Bitcoin")
73     analyze_series("MRF.NS", "MRF Tyres")
74     analyze_series("GC=F", "Gold Futures")

```

② Plain English

This function packages the entire Module 1 exploratory workflow into a single reusable call. Instead of rewriting the same 50 lines three times, you write it once correctly and call it three times. This is the “DRY” principle (Don’t Repeat Yourself) applied to time series analysis.

③ Common Mistake

Hardcoding ticker names in the plot titles and then copying code for each asset — this creates three slightly different versions that drift out of sync when you make improvements. A function ensures all three analyses use identical methodology, making comparisons meaningful.

④ Real World Link

In a quant research team, this kind of “standardized EDA function” is written once and applied to hundreds of assets. If your BTC, MRF, and Gold analyses all use the same function, you know any differences you see in the output are real differences in the assets — not artifacts of inconsistent code.

Section G

Predict the Output

M1.032 *Predict the Output* **EASY**

Topic: Rolling mean NaN count

Without running this code, predict what it will print.

```
1 import pandas as pd
2 import numpy as np
3
4 data = pd.Series([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
5 ma5 = data.rolling(window=5).mean()
6
7 print("Length of ma5:", len(ma5))
8 print("Number of NaN values:", ma5.isna().sum())
9 print("First non-NaN value:", ma5.dropna().iloc[0])
```

Write your predicted output:

Solution

① Predicted Output

```
Length of ma5: 10
Number of NaN values: 4
First non-NaN value: 30.0
```

② Plain English

A rolling window of size 5 needs 5 data points to produce its first result. Points 0–3 (the first 4) don’t have 5 predecessors, so they produce NaN. The first complete window is points 0–4 (values 10, 20, 30, 40, 50), averaging to 30. Total length stays at 10 — pandas preserves the index.

③ Common Mistake

Predicting the length shrinks to 6 (10 - 4). It doesn't — `rolling()` preserves the original index length and fills the unavailable positions with NaN. The length is always the same as the original series. Only `.dropna()` would shorten it to 6.

④ Real World Link

This NaN behavior matters when you overlay a MA on a price chart — the chart simply won't draw the MA line for the first (window-1) days. On a 200-day MA applied to 5 years of BTC data, you lose the first 199 days of MA visibility.

M1.033 Predict the Output **EASY**

Topic: Log return sign

Without running this code, predict what will print.

```
1 import numpy as np
2
3 price_yesterday = 60000
4 price_today     = 57000
5
6 log_return = np.log(price_today / price_yesterday)
7
8 print(f"Log return: {log_return:.4f}")
9 print(f"Positive return? {log_return > 0}")
```

Write your predicted output:

Solution

① Predicted Output

```
Log return:  -0.0513
Positive return?  False
```

② Plain English

$\log(57000/60000) = \log(0.95) \approx -0.0513$. The price fell by 5%, so the log return is negative. Log returns are negative when price falls (ratio < 1) and positive when price rises (ratio > 1).

③ Common Mistake

Computing $\log(\text{price_today}) - \log(\text{price_yesterday})$ separately and expecting an exact match to $\log(\text{price_today}/\text{price_yesterday})$ — they are mathematically identical ($\log(a/b) = \log(a) - \log(b)$). Both are correct.

④ Real World Link

BTC dropped from ~\$60K to ~\$57K in a single day in May 2021. A log return of -0.0513 means a 5% drop. In log-return space, a -5% day for BTC is common and barely noticeable in a long time series.

M1.034 Predict the Output MEDIUM

Topic: ACF at lag 0

Without running this code, predict what it will print.

```
1 import numpy as np
2 import pandas as pd
3 from statsmodels.tsa.stattools import acf
4
5 data = pd.Series([1, 3, 2, 5, 4, 6, 3, 7, 5, 8])
6 acf_values = acf(data, nlags=3)
7
8 print(f"ACF at lag 0: {acf_values[0]:.4f}")
9 print(f"ACF at lag 1: {acf_values[1]:.4f}")
```

Write your predicted output for the first line. For the second line, state whether it will be positive, negative, or zero — and why.

Solution

① Predicted Output

```
ACF at lag 0: 1.0000
ACF at lag 1: [positive value, approximately 0.5--0.8]
```

② Plain English

The ACF at lag 0 is **always exactly 1.0** — it measures the correlation of the series with itself (zero lag), which is perfect correlation by definition. This is true for any series. Lag 1 will be positive because the series shows a general upward trend — consecutive values tend to be similar in direction, creating positive serial correlation.

③ Common Mistake

Expecting lag-0 ACF to be anything other than 1.0. This is a fixed mathematical identity: $\text{Corr}(X_t, X_t) = 1$ always. In the `plot_acf()` function, the lag-0 spike at 1.0 is often removed from the display to make the other lags more readable (use `zero=False` parameter).

④ Real World Link

The near-unit ACF at low lags for BTC prices (not returns) reflects the trend — today's price is highly correlated with yesterday's price. This is the signature of a

random walk and motivates taking first differences before any further analysis.

M1.035 Predict the Output **MEDIUM**

Topic: Exponential smoothing — alpha effect

Given this code, predict which series (s1 or s2) will be smoother, and predict approximate output for the final print.

```

1 import pandas as pd
2
3 data = pd.Series([100, 105, 98, 112, 95, 108, 101, 115,
4                  99, 110])
5
6 def ses(series, alpha):
7     result = [series.iloc[0]]
8     for i in range(1, len(series)):
9         result.append(alpha * series.iloc[i] + (1 - alpha)
10                      * result[-1])
11     return pd.Series(result)
12
13 s1 = ses(data, alpha=0.1)
14 s2 = ses(data, alpha=0.9)
15
16 print(f"s1 variance: {s1.var():.2f}")
17 print(f"s2 variance: {s2.var():.2f}")
18 print(f"Smoother series: {'s1' if s1.var() < s2.var() else
19                          's2'}")

```

Predict the output.

Solution

① Predicted Output

```

s1 variance: [low value, approximately 5--20]
s2 variance: [high value, approximately 25--50]
Smoother series: s1

```

② Plain English

Small alpha (0.1) means heavy weighting on history → very slow to change → low variance (smooth). Large alpha (0.9) means heavy weighting on the latest point → tracks the raw data closely → high variance (wiggly). s1 with alpha=0.1 will be noticeably smoother.

③ Common Mistake

Confusing “smooth” with “accurate.” s2 (alpha=0.9) is more reactive and may track the data better short-term but is not smooth. s1 (alpha=0.1) is smooth but

lags significantly. Smoothness and accuracy involve a tradeoff.

④ Real World Link

A risk manager monitoring MRF's price would prefer small alpha (smooth signal, ignore daily noise) while a day trader wants large alpha (stay close to current price). Same math, completely different use cases — the appropriate alpha depends entirely on your decision-making horizon.

M1.036 *Predict the Output* **HARD**

Topic: Decomposition residual properties

Predict what this code will print. Think carefully about the mathematical properties of additive decomposition.

```
1 import pandas as pd
2 import numpy as np
3 from statsmodels.tsa.seasonal import seasonal_decompose
4
5 np.random.seed(42)
6 t = np.arange(100)
7 seasonal = np.sin(2 * np.pi * t / 12)
8 trend     = 0.05 * t
9 noise     = np.random.normal(0, 0.1, 100)
10 y = pd.Series(trend + seasonal + noise)
11
12 result = seasonal_decompose(y, model="additive", period=12)
13
14 residuals = result.resid.dropna()
15 print(f"Residual mean (approx): {residuals.mean():.3f}")
16 print(f"Residual std (approx): {residuals.std():.3f}")
17 print(f"Residuals near zero? {abs(residuals.mean()) < 0.1}")
```

Solution

① Predicted Output

```
Residual mean (approx): ~0.000 (very close to 0)
Residual std (approx): ~0.1 (approximately equal to noise std)
Residuals near zero? True
```

② Plain English

We constructed the series as `trend + seasonal + noise` and then decomposed it additively. The decomposition correctly recovers the trend and seasonal components,

leaving behind the **noise** term as residuals. Since the noise was $\mathcal{N}(0, 0.1)$, the residuals should be approximately zero-mean with standard deviation around 0.1.

③ Common Mistake

Expecting the residual std to be 0 (perfectly recovered). It won't be exactly 0.1 because the decomposition uses smoothing (moving averages internally) which may absorb a tiny fraction of noise into trend/seasonal. But it will be very close.

④ Real World Link

When you decompose real Gold data, the residual component is everything the model couldn't explain — unexpected events, intervention effects (COVID lockdowns, geopolitical shocks). If residuals have large spikes, those correspond to identifiable real-world events.

Section H

Short Answer

M1.037 Short Answer **EASY**

Topic: White noise definition

In 2 sentences, explain what a white noise process is and give one example of a financial time series that approximately behaves like white noise.

Solution

① Model Answer

A white noise process is a sequence of uncorrelated random variables with constant mean (typically zero) and constant variance — meaning no value provides any information about the next value. Daily log returns of large, liquid assets like BTC or major index stocks often approximate white noise, which is consistent with the weak-form Efficient Market Hypothesis.

② Common Mistake

Saying white noise must be normally distributed — it doesn't have to be. White noise requires only zero autocorrelation (uncorrelated), not Gaussian distribution. BTC returns are approximately white noise in terms of autocorrelation but are clearly non-Gaussian (fat tails, excess kurtosis).

③ Real World Link

If BTC log returns are truly white noise, no ARIMA model can beat a naive “next day = today” forecast — the returns are unpredictable. This is both liberating (you stop wasting time building useless mean models) and frustrating (volatility prediction via GARCH becomes the main value-add instead).

M1.038 *Short Answer* **MEDIUM****Topic:** Additive vs multiplicative model selection

In 2 sentences, explain how you would decide whether to use additive or multiplicative decomposition for a given time series — and identify which you would choose for Bitcoin's price series, with justification.

Solution**① Model Answer**

Use additive decomposition when seasonal fluctuations stay roughly constant in absolute size regardless of the trend level; use multiplicative when seasonal fluctuations grow or shrink proportionally with the trend level. For Bitcoin, multiplicative decomposition is appropriate because price swings at \$60K are much larger in absolute terms than at \$10K — BTC's volatility clearly scales with its price level, a hallmark of multiplicative behavior.

② Common Mistake

Choosing additive for BTC because “the price is just a number.” The key is whether seasonal/cyclical amplitudes scale with price level — not the price magnitude itself. A quick visual check: does the seasonal swing look constant in height regardless of where the series is? If yes → additive. If the swings grow with price → multiplicative.

③ Real World Link

This choice has real consequences for forecast intervals. If you incorrectly use additive decomposition for BTC, your seasonal adjustment during a high-price period will be too small, and your forecast confidence intervals will be underestimated — leading to unpleasant surprises.

M1.039 *Short Answer* **HARD****Topic:** Portmanteau test on model residuals

In 2 sentences, explain why a data analyst applies the Ljung-Box test to the **residuals** of a fitted ARIMA model — not to the original series — and what a significant result means for the model.

Solution**① Model Answer**

After fitting an ARIMA model, the residuals should behave like white noise if the model has captured all systematic structure — applying the Ljung-Box test to residuals is a diagnostic check that verifies this assumption. A significant result ($p < 0.05$) means the residuals still contain autocorrelation, indicating the model is **misspecified** — it has missed some pattern and needs revision (higher AR/MA order, seasonal terms, or a different model class).

② Common Mistake

Applying Ljung-Box only at the data exploration stage and never again during model evaluation. The test on raw returns answers “is there structure to model?” The test on residuals answers “did I successfully model all that structure?” Both questions matter, and many students forget the second one.

③ Real World Link

If you fit an ARIMA(1,1,1) to BTC returns and then run Ljung-Box on the residuals and get $p = 0.003$, your model is leaving money on the table — there’s still autocorrelation your model didn’t capture. You need a higher-order model or should investigate whether GARCH effects are responsible for the remaining structure.

Section I

★ Exam Style

M1.040 Exam Style — Multi-Part EXAM

Topic: Complete Module 1 Analysis on Gold Futures

Time Estimate: 45 minutes — **Total Marks:** 25

This exam-style question requires you to conduct a complete exploratory time series analysis of Gold Futures (GC=F). Read all parts before starting. Each part builds on the previous.

Part A — Data Acquisition & Basic Statistics [4 marks]

Download Gold Futures data (GC=F) from 2019-01-01 to 2024-12-31. Extract the closing price. Print: (i) total number of trading days, (ii) mean closing price, (iii) maximum closing price and its date, (iv) minimum closing price and its date.

Part B — Smoothing & Trend Visualization [5 marks]

On a single plot, display: (i) the raw closing price, (ii) a 20-day MA, (iii) a 100-day MA. Use appropriate colors, labels, and title. In the code comments, explain in one line why longer-window MAs appear smoother.

Part C — Decomposition [5 marks]

Resample to monthly frequency (last price of each month) and apply seasonal decomposition. Use the multiplicative model with appropriate period. Plot all four components. In a comment, state what the trend component reveals about Gold over 2019–2024.

Part D — ACF Analysis [5 marks]

Compute log returns. Plot both the ACF and PACF of the returns (side by side, 40 lags). Comment in code: (i) at approximately which lags you see significant autocorrelation (if any), (ii) what this suggests about predictability of Gold returns.

Part E — Portmanteau Test & Conclusion [6 marks]

Run the Ljung-Box test on the log returns for lags 1 to 20. Print a formatted results table. Write a 3-line comment block that states: (i) your null and alternative hypothesis, (ii) your conclusion based on p-values, (iii) whether Gold returns behave

like white noise and what that implies for forecasting.

Solution

① Complete Solution

```

1  # =====
2  # EXAM SOLUTION -- M1.040 -- Gold Futures Complete Analysis
3  # =====
4  !pip install yfinance statsmodels arch -q
5  import yfinance as yf
6  import numpy as np
7  import pandas as pd
8  import matplotlib.pyplot as plt
9  import warnings
10 warnings.filterwarnings('ignore')
11 from statsmodels.tsa.seasonal import seasonal_decompose
12 from statsmodels.graphics.tsaplots import plot_acf,
    plot_pacf
13 from statsmodels.stats.diagnostic import acorr_ljungbox
14
15 # -----
16 # PART A -- DATA ACQUISITION & BASIC STATISTICS [4 marks]
17 # -----
18 gold = yf.download("GC=F", start="2019-01-01",
19                   end="2024-12-31")
20 close = gold["Close"].dropna()
21
22 print(f"(i)    Total trading days: {len(close)}")
23 print(f"(ii)   Mean closing price: ${close.mean():.2f}")
24 print(f"(iii)  Maximum price: ${close.max():.2f} on
25       {close.idxmax().date()}")
26 print(f"(iv)   Minimum price: ${close.min():.2f} on
27       {close.idxmin().date()}")
28
29 # -----
30 # PART B -- SMOOTHING & TREND VISUALIZATION [5 marks]
31 # -----
32 ma20 = close.rolling(window=20).mean()
33 ma100 = close.rolling(window=100).mean()
34
35 fig, ax = plt.subplots(figsize=(16, 5))
36 ax.plot(close, color="gray", alpha=0.4, linewidth=0.8,
37         label="Gold Close")
38 ax.plot(ma20, color="blue", linewidth=1.5,
39         label="20-Day MA")
40 ax.plot(ma100, color="red", linewidth=2.5,
41         label="100-Day MA")

```

```

36 # Longer windows average over more points, reducing noise
    while increasing lag
37 ax.set_title("Gold Futures -- Price with 20-Day and
    100-Day MAs")
38 ax.legend()
39 plt.tight_layout(); plt.show()
40
41 # -----
42 # PART C -- DECOMPOSITION [5 marks]
43 # -----
44 monthly = close.resample("ME").last()
45
46 # Multiplicative: seasonal amplitude grows proportionally
    with gold price
47 result = seasonal_decompose(monthly,
    model="multiplicative", period=12)
48
49 # Trend shows strong upward move from ~$1300 (2019) to
    ~$2000+ (2024),
50 # driven by COVID-era monetary stimulus and 2024
    geopolitical uncertainty.
51 result.plot()
52 plt.suptitle("Gold Futures -- Multiplicative Seasonal
    Decomposition")
53 plt.tight_layout(); plt.show()
54
55 # -----
56 # PART D -- ACF ANALYSIS [5 marks]
57 # -----
58 returns = np.log(close).diff().dropna()
59
60 fig, axes = plt.subplots(1, 2, figsize=(16, 5))
61 plot_acf(returns, lags=40, alpha=0.05, ax=axes[0])
62 axes[0].set_title("ACF -- Gold Log Returns")
63 plot_pacf(returns, lags=40, alpha=0.05, ax=axes[1])
64 axes[1].set_title("PACF -- Gold Log Returns")
65
66 # (i) Significant autocorrelation at small number of lags
    (typically lags 1, 6, 15)
67 # (ii) Most lags within confidence bands -- Gold returns
    are APPROXIMATELY white
68 #     noise but not purely so -- mild autocorrelation
    limits predictability
69
70 fig.suptitle("Gold Futures -- ACF and PACF of Log
    Returns", fontsize=13)
71 plt.tight_layout(); plt.show()
72
73 # -----

```



```

74 # PART E -- PORTMANTEAU TEST & CONCLUSION [6 marks]
75 # -----
76 lb_result = acorr_ljungbox(returns, lags=20,
77                             return_df=True)
78
79 print("\n" + "="*55)
80 print("  Ljung-Box Test Results -- Gold Log Returns")
81 print("="*55)
82 print(f"    {'Lag':>4}    {'LB Stat':>10}    {'p-value':>10}
83         {'Sig?':>6}")
84 print("-"*55)
85 for lag, row in lb_result.iterrows():
86     sig = "YES" if row["lb_pvalue"] < 0.05 else "no"
87     print(f"    {lag:>4}    {row['lb_stat']:>10.4f}"
88           f"    {row['lb_pvalue']:>10.4f}    {sig:>6}")
89 print("="*55)
90
91 # H0: Gold log returns are white noise (no autocorrelation
92 #    up to lag 20)
93 # H1: Gold log returns are NOT white noise (significant
94 #    autocorrelation exists)
95 # Conclusion: If any p-value < 0.05, reject H0 -- returns
96 #    show predictable structure.
97 # Gold returns are CLOSE TO but NOT PERFECTLY white noise
98 #    -- most predictability
99 # comes from volatility (GARCH effects), not mean
100 # autocorrelation, making
101 # Module 4 (GARCH) the more valuable tool for Gold price
102 # modelling.
103
104 n_sig = (lb_result["lb_pvalue"] < 0.05).sum()
105 print(f"\nSignificant lags at 5% level: {n_sig} out of 20")
106 if n_sig >= 3:
107     print("-> Reject white noise -- significant
108           autocorrelation found")
109 else:
110     print("-> Cannot conclusively reject white noise --
111           returns largely unpredictable")

```

② Plain English

This exam question walks through the complete Module 1 workflow in a single coherent analysis: get data → describe it → smooth it → decompose it → check autocorrelation → test for white noise. Each part produces both a visual output and a quantitative result. The final conclusion ties statistical findings back to practical forecasting implications.

③ Common Mistake — Part C

Using `model="additive"` for Gold without justification. Since Gold price roughly tripled from 2019 to 2024, seasonal swings are proportionally larger at higher prices

— this is the definition of multiplicative behavior. Exam answers must justify model choice, not just state it.

④ Real World Link

This exact analysis — download, smooth, decompose, ACF, Ljung-Box — is conducted by every commodity research desk before building any Gold price forecast model. The Ljung-Box result on Gold returns typically shows that volatility (GARCH) modeling is more valuable than mean modeling for Gold, which is why this module's findings directly motivate the GARCH models in Module 4.

Module 1 Complete

40 Problems — All 8 Types — All 4 Difficulty Levels

Type	Problems	Count
Fill in the Blank	M1.001–M1.006	6
Find the Error	M1.007–M1.012	6
Get the Order Right	M1.013–M1.016	4
Match the Following	M1.017–M1.020	4
Multiple Choice	M1.021–M1.026	6
Write from Scratch	M1.027–M1.031	5
Predict the Output	M1.032–M1.036	5
Short Answer	M1.037–M1.039	3
Exam Style	M1.040	1
Total		40

Ready for **Module 2** — Stationary Models: AR, MA, ARMA, ARIMA, SARIMA